

The entire code is available at <https://github.com/SupriyoGanguly/Linux-Firewall-by-netfilter>. You can download the code files to check and understand the implementation.

Packet filtering

We have created a *firewall.c* file that is available in the download from the above link. In *firewall.c*, the *netfilter_ops* is a *struct nf_hook_ops* variable. In the init-module section, *netfilter_ops* is initialised with the following:

```
netfilter_ops.hook      =      main_hook; //the
handler function
netfilter_ops.pf        =      PF_INET; //tells the
Protocol is IPv4
netfilter_ops.hooknum   =      NF_INET_POST_ROUTING;
//process at post-routing stage
netfilter_ops.priority  =      NF_IP_PRI_FIRST; //
priority
```

Given below is the snippet from *firewall.c*:

```
static struct nf_hook_ops netfilter_ops;

/* This function is called by hook. */

static unsigned int main_hook(void *priv, struct sk_buff
*skb, const struct nf_hook_state *state)
{
    //struct udphdr *udp_header;
    int dstPort;
    struct tcphdr *hdr;
    struct iphdr *ip_header = (struct iphdr *)skb_network_
header(skb);

    if (ip_header->protocol == IPPROTO_ICMP) {
        // udp_header = (struct udphdr *)skb_transport_
header(skb);

        printk(KERN_INFO "Drop icmp packet.\n");
        return NF_DROP;
    }

    if (ip_header->protocol == IPPROTO_TCP) {
        hdr = (struct tcphdr *) skb_transport_header(skb);
        dstPort = ntohs(hdr->dest);
        if((dstPort==443) || (dstPort==80)) /*drop https and
http*/ {
            printk("Drop HTTPS/HTTP packet\n");
            return NF_DROP;
        }
    }
    return NF_ACCEPT;
}
```

main_hook is the name of the handler function for the *NF_INET_POST_ROUTING* hook. In this function, any packet (whether forwarded or sent from the local interface with ICMP protocol) will be dropped using the first 'if' statement, where it checks for the IP_PROTOCOL in the IPv4 header of the socket buffer. As the hook is returning *NF_DROP*, it tells the kernel driver not to proceed with the packet.

The second 'if' statement checks whether any TCP packet with destination port number 443 (for HTTPS) or with port number 80 (for HTTP) will be dropped.

Now we can use the *Make* statement to compile this module (*firewall.c*). Figure 2 shows that we can ping an IP of a local network device with IP address 192.168.29.1 successfully just before we have implemented *firewall.c*.

But after insertion of the module, ping starts to fail (as shown in Figure 3).

```
supriyo@breakout: ~/Documents
File Edit View Search Terminal Help
supriyo@breakout:~/Documents$ ping 192.168.29.1
PING 192.168.29.1 (192.168.29.1) 56(84) bytes of data.
64 bytes from 192.168.29.1: icmp_seq=1 ttl=64 time=2.40 ms
64 bytes from 192.168.29.1: icmp_seq=2 ttl=64 time=4.33 ms
64 bytes from 192.168.29.1: icmp_seq=3 ttl=64 time=1.48 ms
64 bytes from 192.168.29.1: icmp_seq=4 ttl=64 time=16.0 ms
64 bytes from 192.168.29.1: icmp_seq=5 ttl=64 time=1.45 ms
64 bytes from 192.168.29.1: icmp_seq=6 ttl=64 time=2.69 ms
64 bytes from 192.168.29.1: icmp_seq=7 ttl=64 time=5.82 ms
```

Figure 2: Successful ping

```
supriyo@breakout: ~/Documents
File Edit View Search Terminal Help
supriyo@breakout:~/Documents$ sudo insmod firewall.ko
[sudo] password for supriyo:
supriyo@breakout:~/Documents$ ping 192.168.29.1
PING 192.168.29.1 (192.168.29.1) 56(84) bytes of data.
^C
--- 192.168.29.1 ping statistics ---
23 packets transmitted, 0 received, 100% packet loss, time 22524ms
```

Figure 3: Unsuccessful ping

```
supriyo@breakout: ~/Documents
File Edit View Search Terminal Help
119.596120] usb 1-7: reset high-speed USB device number 4 using ehci-pci
240.207935] firewall: loading out-of-tree module taints kernel.
240.207993] firewall: module verification failed: signature and/or required k
ey missing - tainting kernel
240.208126] Init MY MODULE
240.208127] Registering for 3
240.208130] Registering for 1
240.208132] Registering for 1
247.795350] Drop icmp packet.
255.559446] Drop icmp packet.
256.579776] Drop icmp packet.
257.603772] Drop icmp packet.
258.637745] Drop icmp packet.
259.651725] Drop icmp packet.
260.675725] Drop icmp packet.
261.699704] Drop icmp packet.
262.723683] Drop icmp packet.
263.747672] Drop icmp packet.
263.785479] Drop HTTPS/HTTP packet
264.771658] Drop icmp packet.
264.794756] Drop HTTPS/HTTP packet
265.795644] Drop icmp packet.
266.804207] Drop HTTPS/HTTP packet
```

Figure 4: Output of *dmesg*